PM2.5 over India

Machine learning for spatial enhancement of air-quality maps

A beginner's guide to the whole project:
the data, the 12 steps, every Python file, and what we found

Course project 3 · CAMS EAC4 reanalysis · India · 2020–2024

Repository: github.com/dineman/pm25-india · Booklet written October 2026

Contents

How to read this booklet . . . 1
1 What is PM2.5, and why map it? . . . 3
2 Where the data comes from . . . 3
2.1 CAMS and “reanalysis” . . . 3
2.2 The grid . . . 3
2.3 The variables . . . 4
2.4 How much data . . . 4
3 What is machine learning? . . . 4
4 The questions this project asks . . . 5
4.1 The tools used . . . 5
5 Step 01: Downloading the data . . . 8
6 Step 02: Which grid cells are “India”? . . . 8
7 Step 03: Cleaning and preparing the data . . . 9
8 Step 04: Exploring the data . . . 10
9 Step 05: Splitting India into training and test areas . . . 11
10 Step 06: Simple methods first (baselines) . . . 12
11 Step 07: Machine-learning models . . . 13
12 Step 08: Combining ML and interpolation (the hybrid) . . . 13
13 Step 09: Careful evaluation . . . 14
14 Step 10: Which variables matter? . . . 15
15 Step 11: Making finer maps . . . 16
16 Step 12: Writing it up . . . 17
17 The answers in short . . . 18
18 How the repository is organised . . . 20
19 The library: `src/pm25/` . . . 21
19.1 `download.py`: requesting data from Copernicus . . . 21
19.2 `regions.py`: India mask and regions . . . 21
19.3 `preprocess.py`: units and derived variables . . . 21
19.4 `exploration.py`: summaries, correlations, variograms . . . 21
19.5 `splits.py`: blocks, test set and folds . . . 22
19.6 `features.py`: from grids to tables . . . 22
19.7 `metrics.py`: how every method is scored . . . 22
19.8 `baselines.py`: interpolation as weight matrices . . . 23
19.9 `models.py`: the ML models and tuning . . . 23
19.10 `hybrid.py`: regression kriging . . . 23
19.11 `evaluation.py`: bootstrap and CV check . . . 24
19.12 `importance.py`: SHAP and permutation importance . . . 24
19.13 `maps.py`: the 0.25° maps . . . 24
19.14 `logging_utils.py`: recording every run . . . 24
20 The scripts: `scripts/` . . . 25
21 The tests: `tests/` . . . 26
22 Documentation and record keeping . . . 28
23 Running the project yourself . . . 30
23.1 What you need . . . 30
23.2 Setting up . . . 30

23.3 Running the steps 30
23.4 Where to find things afterwards 31
24 Mistakes we made and what they taught us 31
25 What the results can and cannot tell us 32
26 Glossary 32
26.1 Data 32
26.2 Statistics and spatial analysis 33
26.3 Splitting and evaluation 33
26.4 Interpolation (baselines) 34
26.5 Machine learning (step 07) 35
26.6 Feature importance (step 10) 35
26.7 Maps (step 11) 36

How to read this booklet

This booklet explains a complete data-science project from the very beginning. You do **not** need to know air pollution science, statistics or machine learning to follow it: every idea is introduced in plain words the first time it appears, and the most important terms are collected in the glossary at the end.

The booklet has four parts:

- **Part I, The big picture** (chapters 1–4): what PM2.5 is, where the data comes from, what machine learning is, and which questions the project asks.
- **Part II, The project step by step** (chapters 5–17): the twelve steps of the work in the order they were done, each with *what* was done, *why*, *how*, and *what we found*, followed by the answers in short.
- **Part III, The code** (chapters 18–22): a guided tour of the repository and of **every Python file**, with short pieces of code explained line by line.
- **Part IV, Using and judging the project** (chapters 23–26): how to run it yourself, the mistakes we found and fixed, the limits of the results, and the glossary.

If you are in a hurry, read chapter 4 (the questions) and chapter 17 (the answers). If you want to change the code, read Part III.

Each step chapter starts with an **"In one sentence"** line that tells you the main idea before the details.

A note on numbers: PM2.5 is measured in **micrograms per cubic metre of air**, written **µg m⁻³** (a microgram is a millionth of a gram). India's national 24-hour limit is 60 µg m⁻³; the World Health Organization's guideline is 15 µg m⁻³.

Part I

The big picture

1 What is PM2.5, and why map it?

PM2.5 means *particulate matter smaller than 2.5 micrometres*: tiny solid or liquid particles floating in the air, about 30 times thinner than a human hair. They come from burning (vehicles, industry, power plants, cooking and heating fires, crop-residue burning, forest fires), from dust, and from chemical reactions of gases in the air. Because they are so small, they get deep into the lungs and even into the blood. Long exposure to PM2.5 is one of the largest environmental health risks in the world, and India has some of the highest levels anywhere, especially across the plains of the north in winter.

To protect people you need to know **how much PM2.5 there is, where and when**. Ground stations measure it precisely, but there are far too few of them to cover a country of 3.3 million km². This is where **maps** come in: a map estimates PM2.5 everywhere, including places without a station.

Our project builds such maps with **machine learning**: a computer learns the relationship between PM2.5 and things we know everywhere (such as weather and the haziness of the sky seen from space), and then uses that relationship to estimate PM2.5 at places where it was not given the answer.

2 Where the data comes from

2.1 CAMS and "reanalysis"

We use **CAMS EAC4**, a dataset from the European *Copernicus Atmosphere Monitoring Service*. EAC4 is a **reanalysis**: a computer model of the atmosphere (weather plus chemistry) is run over the past years, and real satellite observations are continuously fed into it to keep it close to reality. The result is a complete, gap-free record of the atmosphere, every 3 hours, on a regular grid covering the whole globe.

Important: CAMS is **a model, not measurements**. When this booklet says a method is "accurate", it means "it agrees well with CAMS", not "it agrees with real air quality". We come back to this in chapter 25.

2.2 The grid

The Earth's surface is divided into squares of **0.75° × 0.75°** of latitude and longitude, about **80 km × 80 km** over India. Each square is a **grid cell**. CAMS gives one value per cell for each variable at each time step. Our box around India (6–38° N, 68–98° E) contains 43 × 40 = 1,720 cells, of which **494 lie (mostly) inside India**.

2.3 The variables

Variable	What it is	Role
PM2.5	particle mass, μg m^{-3}	what we want to predict (the *target*)
AOD (aerosol optical depth)	how much the particles in the whole column of air dim sunlight; a satellite-style "haziness" number	input (*predictor*)
u10, v10	wind towards the east and towards the north, 10 m above ground	input
t2m	air temperature 2 m above ground	input
d2m	dew point 2 m above ground (a measure of moisture)	input
sp	surface pressure; low pressure means high altitude	input
latitude, longitude	where the cell is	input

From these we also compute **wind speed**, **relative humidity** and the **time of day** and **time of year** (chapter 7).

2.4 How much data

Five years, 2020–2024, every 3 hours: **14,616 time steps**. For the 494 India cells that is **7.2 million** rows of data, each with PM2.5 and all predictors.

3 What is machine learning?

Machine learning (ML) is a way to make a computer find a rule from examples instead of a person writing the rule. We show the computer millions of examples of the form *"here are the weather and AOD at this place and time → and this was the PM2.5"*. A learning algorithm adjusts a *model* until its guesses match the examples as well as possible. Afterwards we can give the model new weather and AOD values and it **predicts** PM2.5.

Three ideas matter throughout this project:

1. **Training and testing.** A model is always judged on examples it did **not** learn from (the *test set*). Scoring a model on its own training examples is like grading a student with the exact questions they practised: it says nothing about whether they understood.
2. **Different kinds of models.** Some are simple (a *straight-line* formula), some are flexible (*decision trees*, *neural networks*). Flexible models can capture complicated patterns but can also "memorise" noise. We compare several (chapter 11).
3. **Interpolation is the old alternative.** Without ML, the classic way to fill a map is to **interpolate**: estimate a place from its measured neighbours (e.g. average the nearest ones).

The project's central question is whether ML does better than this, and whether combining both is best.

4 The questions this project asks

The course brief asks six **research questions (RQ)**:

1. **How accurately** can PM2.5 be estimated from CAMS atmospheric and aerosol variables?
2. **How do different ML models compare?**
3. **Which variables matter most** for the prediction?
4. **How does performance change across regions and seasons** of India?
5. **How well do the models work at places they were not trained on?**
6. **How does ML-based "spatial enhancement" compare with standard interpolation?**

"Spatial enhancement" means making a **finer map** than the 80 km CAMS grid.

The single most important design decision of the whole project follows from question 5: **all testing is done at grid locations that were completely hidden during training**, and those hidden locations are whole *blocks* of neighbouring cells, not scattered single cells. Chapter 9 explains why this matters so much.

4.1 The tools used

The project is written in **Python**, the most common language for data science. It relies on well-known free libraries:

Library	Used for
NumPy	fast arrays of numbers and maths on them
pandas	tables (rows and columns), grouping, CSV files
xarray	labelled multi-dimensional data (time × lat × lon) and NetCDF files
scikit-learn	standard ML models (ridge, random forest, neural network) and tools
LightGBM	fast gradient-boosted decision trees, our best model
SciPy, **PyKrige**	triangulation and a kriging check
Shapely, **Cartopy**	geometry (country/state shapes) and map outlines
matplotlib	all figures
cdsapi	downloading from the Copernicus data store
pytest, **ruff**	automatic tests and code-style checking

Everything is installed with **conda** from one file, `environment.yml`, so the whole environment can be rebuilt on any computer. The project's history is kept with **git** and stored on **GitHub**.

It was worked on from two computers (a Fedora Linux machine and a Mac), and git kept them in sync.

Part II

The project step by step

5 Step 01: Downloading the data

In one sentence: fetch five years of CAMS data for India from the internet, one month at a time, in a way that survives interruptions.

What and why. CAMS data lives on the Copernicus *Atmosphere Data Store* (ADS). You send it a *request* (which variables, which days, which hours, which area), the request waits in a queue, and then you download a file. We need seven variables for 60 months.

How. One request per month keeps each job small. The script:

- skips a month whose file already exists, so a crashed run can simply be started again;
- downloads to a temporary name (`….nc.part`) and renames the file only when it is complete, so a half-downloaded file is never mistaken for a finished one;
- logs every failure and moves on, listing failed months at the end.

What happened. All 60 files (381 MB in total, NetCDF format) arrived after about **5¾ hours**, slower than expected because the ADS servers kept dropping connections; the download client retried automatically.

Code: `src/pm25/download.py`, `scripts/download_eac4.py` (chapter 19).

6 Step 02: Which grid cells are "India"?

In one sentence: decide, for every 80 km grid cell, whether it belongs to India and to which of six regions.

Why. The download box also contains sea, Pakistan, Nepal, Tibet, Bangladesh and Myanmar. Their pollution has different sources, so the project uses only Indian cells. We also need **regions**, because question 4 asks how results vary across India.

How. We take the outlines of India's states from *Natural Earth* (a free map dataset), and for each grid cell compute what fraction of its area lies inside India and which state covers most of it. A cell counts as India if **at least 50%** of it is inside. Each state belongs to one of six regions:

Region	States / union territories
IGP (Indo-Gangetic Plain)	Punjab, Haryana, Delhi, Chandigarh, Uttar Pradesh, Bihar, West Bengal
northwest	Rajasthan, Gujarat, Dadra & Nagar Haveli and Daman & Diu
central	Madhya Pradesh, Chhattisgarh, Jharkhand, Odisha, Maharashtra
south	Andhra Pradesh, Telangana, Karnataka, Goa, Kerala, Tamil Nadu, Puducherry
northeast	Assam, Arunachal Pradesh, Meghalaya, Manipur, Mizoram, Nagaland, Tripura, Sikkim
Himalaya	Jammu & Kashmir, Ladakh, Himachal Pradesh, Uttarakhand

What we found. 494 cells are India: IGP 84, northwest 83, central 150, south 97, northeast 43, Himalaya 37. The 50% rule leaves out 127 border and coast cells, including the cells of **Mumbai** (30% inside India) and **Chennai** (47%); this was discussed and kept as a known limitation. Delhi's cell is labelled Haryana (Haryana covers more of it), which doesn't matter because both are in the IGP region.

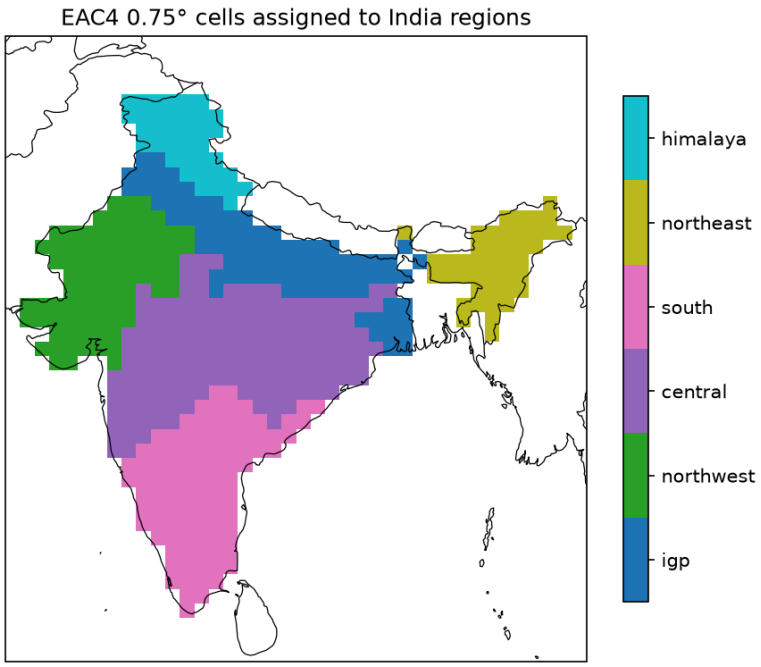


Figure 1: The 494 India cells and their regions

Code: `src/pm25/regions.py`, `scripts/build_region_mask.py`.

7 Step 03: Cleaning and preparing the data

In one sentence: join the 60 monthly files into one tidy dataset with sensible units and some extra, derived variables.

How.

1. **Tidy names and order**: dimensions are called `time`, `lat`, `lon`; latitude runs south to north.
2. **Units**: PM2.5 from kg m^{-3} to **µg m^{-3}** ($\times 10^9$); temperatures from Kelvin to **°C**; pressure from Pa to **hPa**.
3. **Derived variables**:
   - **wind speed** = $\sqrt{(u10^2 + v10^2)}$ (Pythagoras on the two wind components);
   - **relative humidity** from temperature and dew point (Magnus formula);
   - **local solar hour**: the clock time set by the Sun at that longitude (UTC + longitude/15 hours), because the daily cycle of pollution follows the Sun, not the time zone;
   - **day of year**.
4. **Cyclic encoding**: hour and day of year are written as *sine and cosine* pairs. Why? As plain numbers, 23:00 and 01:00 look far apart (23 vs 1), but they are only two hours apart; on a circle (sin, cos) they are neighbours.
5. **Seasons** as defined by the India Meteorological Department (IMD): winter (Jan–Feb), pre-monsoon (Mar–May), monsoon (Jun–Sep), post-monsoon (Oct–Dec).

What we found. 14,616 time steps, every gap exactly 3 hours, **no missing values**. PM2.5 over India: mean 72, median 52, maximum **5,651 µg m⁻³** (Assam, March 2021, a fire). 2020 had the lowest yearly mean (66 µg m⁻³), consistent with the COVID-19 lockdown. The final file is 632 MB.

Code: `src/pm25/preprocess.py`, `scripts/preprocess_eac4.py`.

8 Step 04: Exploring the data

In one sentence: look at the data from every angle before modelling, to understand its patterns and to choose how to test the models fairly.

Patterns in PM2.5.

- The **Indo-Gangetic Plain is 2–4 times more polluted** than the rest of India in winter (mean 218 µg m⁻³) and post-monsoon (207).
- The **northeast peaks in March** (fire season); no other region does.
- Every region is **cleanest in the monsoon** (rain washes particles out), except that the northwest stays fairly high (desert dust).
- A strong **daily cycle**: highest at night and early morning (02:30–05:30 Indian time), lowest in the afternoon (14:30). At night the lowest layer of the atmosphere (the *boundary layer*) is shallow and traps pollution; in the afternoon the Sun heats the ground and mixes pollution upwards.

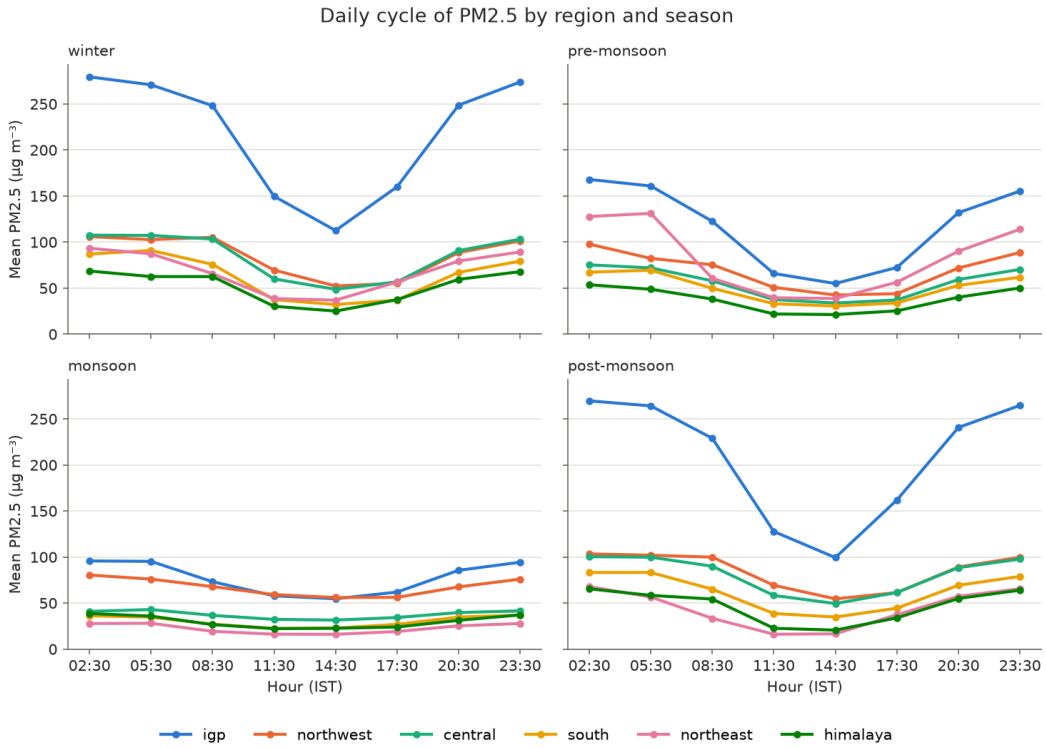


Figure 2: Daily cycle of PM2.5 by region and season

Which variables go with PM2.5? We used the *Spearman correlation*, a number from −1 to +1 telling how consistently two things rise and fall together. AOD has the strongest link (0.60). Surface pressure follows (0.43), because it mostly measures altitude: low plains are polluted,

mountains are clean. Humidity matters only in winter (foggy nights), and temperature even **changes sign** between seasons (−0.30 in winter, +0.34 in the monsoon). Such season-dependent effects are hard for simple straight-line models.

How far does "similarity" reach? A **variogram** measures how different two cells are, on average, as a function of the distance between them. Close cells are similar; far cells are not. After removing the effect of the predictors and of a smooth north–south trend, cells only **75 km apart already differ by 30%** of the maximum, and the difference levels off at about **900 km** (the *range*). This number decides the size of the test blocks in step 05.

Code: `src/pm25/exploration.py`, `scripts/explore_eac4.py`.

9 Step 05: Splitting India into training and test areas

In one sentence: hide whole squares of neighbouring cells from the models, so that testing measures performance at genuinely new places.

Why blocks? Neighbouring cells are almost alike. If we hid single random cells, every hidden cell would have near-copies right next to it in the training data, and any method would look excellent. That would not tell us how well the model works at a place far from known data. So we hide **blocks** of cells.

Choosing the block size. The variogram range (~900 km) would mean blocks of 12 × 12 cells, but India then contains only 11 blocks; a 20% test set would be 2 blocks, far too few. We chose **5 × 5 cells (~380 km)**, where about 75–80% of the local similarity has disappeared, giving 33 blocks.

How the test blocks were picked. A random search tried 5,000 combinations of blocks and kept the one where every region has close to 20% of its cells in the test set. The remaining training blocks were dealt into **5 folds** (groups) for cross-validation (step 07).

Result. 95 test cells in 6 blocks (19%) and **399 training cells in 27 blocks**. Train and test have the same average PM2.5 (72.1 vs 72.6 μg m^{-3}). Test cells are a median 83 km from the nearest training cell. A second split with 8 × 8 blocks was made as a sensitivity check.

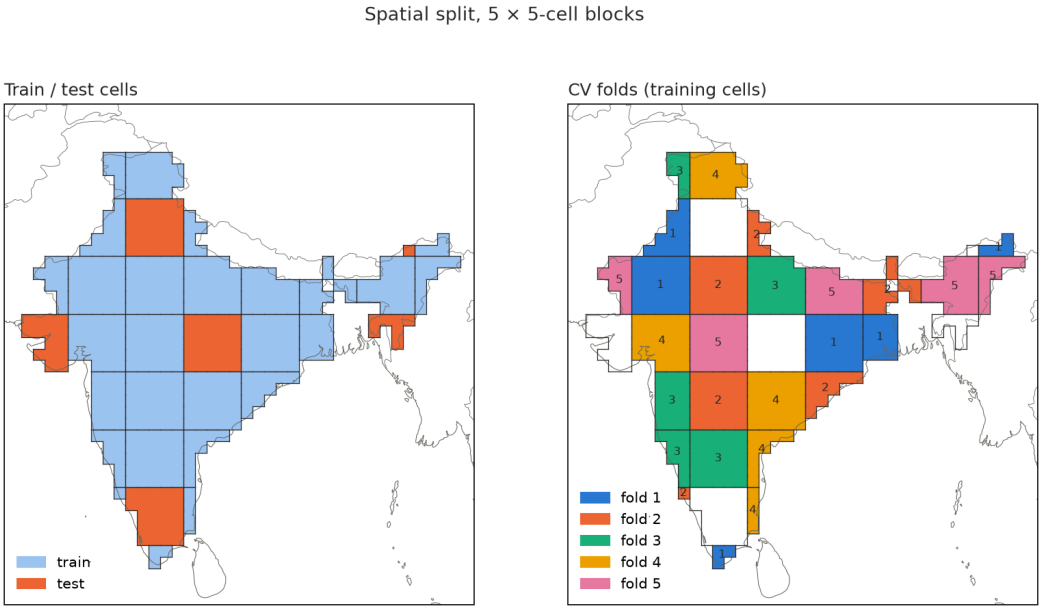


Figure 3: Train/test blocks (left) and cross-validation folds (right)

Code: `src/pm25/splits.py`, `scripts/make_splits.py`.

10 Step 06: Simple methods first (baselines)

In one sentence: before using ML, measure how well simple, classic methods do, so we know what "good" means.

The baselines.

- **Nearest neighbour**: copy the value of the closest training cell.
- **IDW** (*inverse distance weighting*): a weighted average of the 8 nearest training cells, closer ones counting more.
- **Delaunay-linear**: connect training cells into triangles and blend the three corners (the scattered-point version of bilinear interpolation).
- **Ordinary kriging**: a weighted average whose weights come from the variogram, so they reflect how quickly similarity fades with distance.
- **Linear regression**: a straight-line formula predicting PM2.5 from the predictors at the cell itself (no neighbours used).

How we score. Four numbers, used everywhere in the project:

- **RMSE** (root mean squared error): typical error size, punishing big errors heavily, in μg m^{-3};
- **MAE** (mean absolute error): typical error size, treating all errors alike;
- **R^2**: share of the variation explained; 1 is perfect, 0 is no better than always guessing the average;
- **bias**: average of (prediction − truth); negative means "predicts too low".

What we found (test cells). Nearest neighbour R^2 0.61, IDW 0.59, kriging 0.58, linear regression only **0.30**. The straight-line model is poor because the relationships are curved and change with the season (step 04). Taking the logarithm of PM2.5 for linear regression made things worse: once transformed back, one prediction reached 55,041 μg m^{-3}. That lesson led to **clipping** predictions to a sensible range in all later models.

Code: `src/pm25/baselines.py`, `src/pm25/metrics.py`, `scripts/run_baselines.py`.

11 Step 07: Machine-learning models

In one sentence: train four kinds of ML model under identical conditions and see which predicts PM2.5 best at the hidden test cells.

The four models.

Model	Idea in plain words
Ridge	a straight-line formula with a penalty that keeps it from over-reacting
Random forest	hundreds of *decision trees* (flowcharts of yes/no questions such as "is AOD > 0.5?"), each built on a random sample of the data; their answers are averaged
LightGBM	*gradient boosting*: trees built one after another, each correcting the mistakes of the ones before
MLP	a small *neural network*: layers of simple units connected by learned weights

Fair tuning. Every model has *hyperparameters* (settings chosen before training, such as the number of trees). Each model got the **same budget**: 8 random settings, each scored by **5-fold block-wise cross-validation** on the training cells only. Cross-validation means: train on 4 folds, test on the 5th, repeat 5 times, average. The test cells were not touched until the very end. Each model could also choose to learn **log(1 + PM2.5)** instead of PM2.5; this squeezes the huge range of values. All three flexible models chose the log.

What we found (test cells). LightGBM **R^2 0.59** (MAE 19.1, the lowest of all methods), MLP 0.53, random forest 0.47, ridge 0.31. ML beats interpolation far from training cells and in the northwest and Himalaya; interpolation wins in the northeast (fires). Removing latitude/longitude lowers LightGBM to 0.48: location carries large-scale information (where emissions are high) that the weather variables can't.

Code: `src/pm25/models.py`, `src/pm25/features.py`, `scripts/train_models.py`.

12 Step 08: Combining ML and interpolation (the hybrid)

In one sentence: let the ML model predict, then correct it with the errors it made at nearby known cells.

The idea. Where the ML model is wrong at a training cell, it is probably wrong in the same direction at nearby cells too (for example, a city's extra emissions that the weather variables don't describe). So: (1) compute the model's errors (*residuals*) at the training cells, (2) interpolate those errors to the test cell by kriging, (3) add them to the ML prediction. This classic method is called **regression kriging**.

A subtle point. The residuals at training cells must come from a model that **did not see** those cells (*out-of-fold* predictions), otherwise they are unrealistically small. We used the 5 folds again: each fold's residuals come from a model trained on the other four.

What we found (test cells). R^2 rose from 0.59 (ML) and 0.58 (kriging) to **0.72**; the bias dropped from −7.9 to −1.4 μg m^{-3}. Step 09 checks whether this gain is real or partly luck.

Code: `src/pm25/hybrid.py`, `scripts/run_hybrid.py`.

13 Step 09: Careful evaluation

In one sentence: compare all methods in one place, measure how uncertain the scores are, and re-check the conclusions on many more hidden blocks.

The problem. The test set is only **6 blocks**, and most regions are a single block. One unusual block can decide a comparison.

Two remedies.

1. **Block bootstrap.** Pretend we had drawn a different set of 6 test blocks: draw 6 blocks *with replacement* from the 6 we have (some twice, some not at all), recompute every score, and repeat 2,000 times. The spread of the results is a **95% range** for each score and for the difference between two methods.
2. **CV check.** Re-run the comparison on the 399 *training* cells: predict each fold only from the other four folds. That gives **27 blocks** spread over all regions. For the hybrid, the residuals must come from models that never saw the fold being predicted (*nested* cross-validation, 20 extra model fits).

What we found.

- Over 27 blocks: hybrid **R^2 0.68**, LightGBM 0.66, kriging 0.66, nearest 0.56.
- The hybrid's big test gain came almost entirely from **2 of the 6 test blocks**; over 27 blocks it helps in 15 and hurts in 12. Its overall advantage is small and uncertain.
- Results true on both sets: ML beats interpolation in the **Himalaya** (interpolation fails there) and 100–300 km from known cells; the hybrid **removes LightGBM's tendency to predict too low** and **halves the under-prediction of severe pollution**; kriging the residuals beats IDW.
- Errors are **4–5 times larger at night** than in the afternoon, and largest in the northeast and the pre-monsoon (fires).

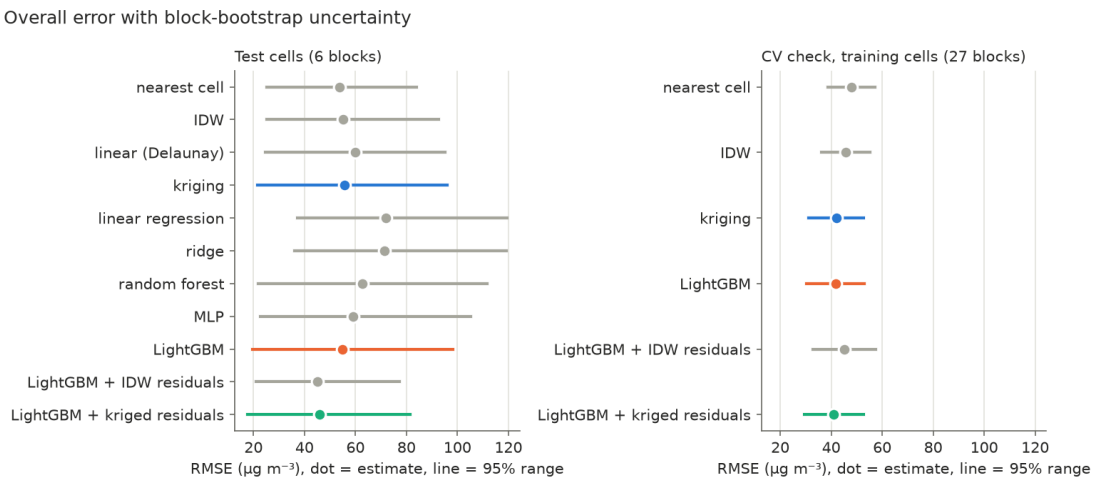


Figure 4: Overall error with 95% ranges: 6 test blocks (left) vs 27 CV blocks (right)

Code: `src/pm25/evaluation.py`, `scripts/evaluate.py`.

14 Step 10: Which variables matter?

In one sentence: open up the LightGBM model and measure how much each variable drives its predictions and its accuracy.

Two tools.

- **SHAP values** split each single prediction into contributions, one per variable ("AOD pushed this prediction up 30%, the hour pulled it down 10%"). Averaging their size gives each variable's importance.
- **Permutation importance**: shuffle one variable's values between rows so it no longer matches the rest, and see how much worse the predictions get. Related variables (temperature, dew point, humidity) are shuffled together.

What we found. AOD is the variable the model uses most (typical effect ±35–40%), then **time of day**, **time of year**, **location** and **altitude** (surface pressure); **wind** matters least. Location matters most for accuracy at new places. Predictions are ~30% higher at night and ~25% lower at midday; cold raises PM2.5 only in winter and post-monsoon; in the Himalaya, altitude and AOD dominate.

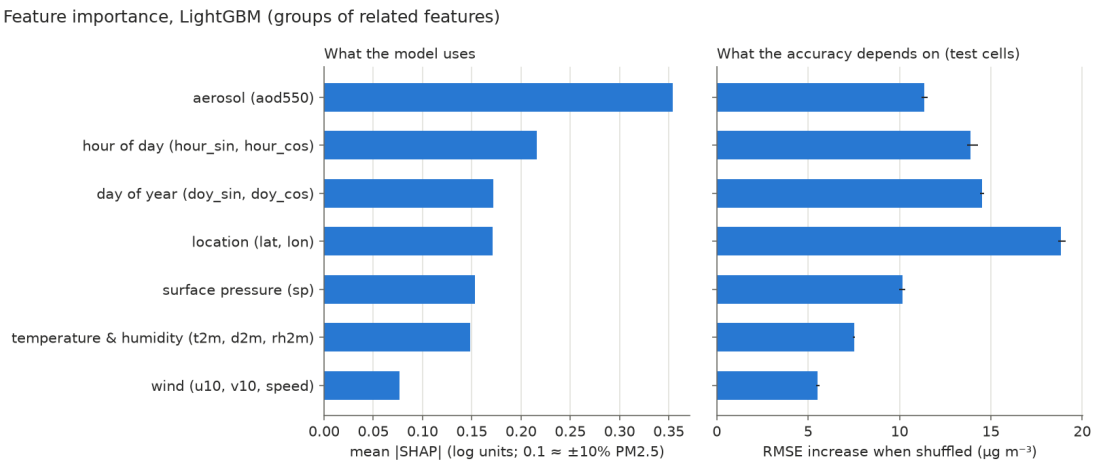


Figure 5: What the model uses (left) and what its accuracy depends on (right)

Code: `src/pm25/importance.py`, `scripts/feature_importance.py`.

15 Step 11: Making finer maps

In one sentence: use the models to draw PM2.5 maps at 0.25° (~28 km), three times finer than CAMS, and compare them with simple regridding.

How. Make a fine grid of 4,636 points inside **India's official boundary** (including all of Jammu & Kashmir and Ladakh). At each point, estimate the predictors by bilinear interpolation of the CAMS fields, compute humidity, wind speed and the time features exactly as in step 03, and let LightGBM predict. The hybrid adds kriged model errors from the 494 CAMS cells. For comparison, CAMS PM2.5 is also regridded bilinearly (the standard method) and by kriging.

What we found. All maps show the same big picture. The hybrid stays almost as faithful to CAMS as bilinear regridding (averaged back to 0.75°: R^2 0.98 vs 0.985), while LightGBM alone drifts away (0.84). The hybrid adds a little extra detail at the plain-to-Himalaya edge, where it knows the altitude. But no data exists below 0.75° to check the finer details: they come from what the model has learned, not from new measurements.

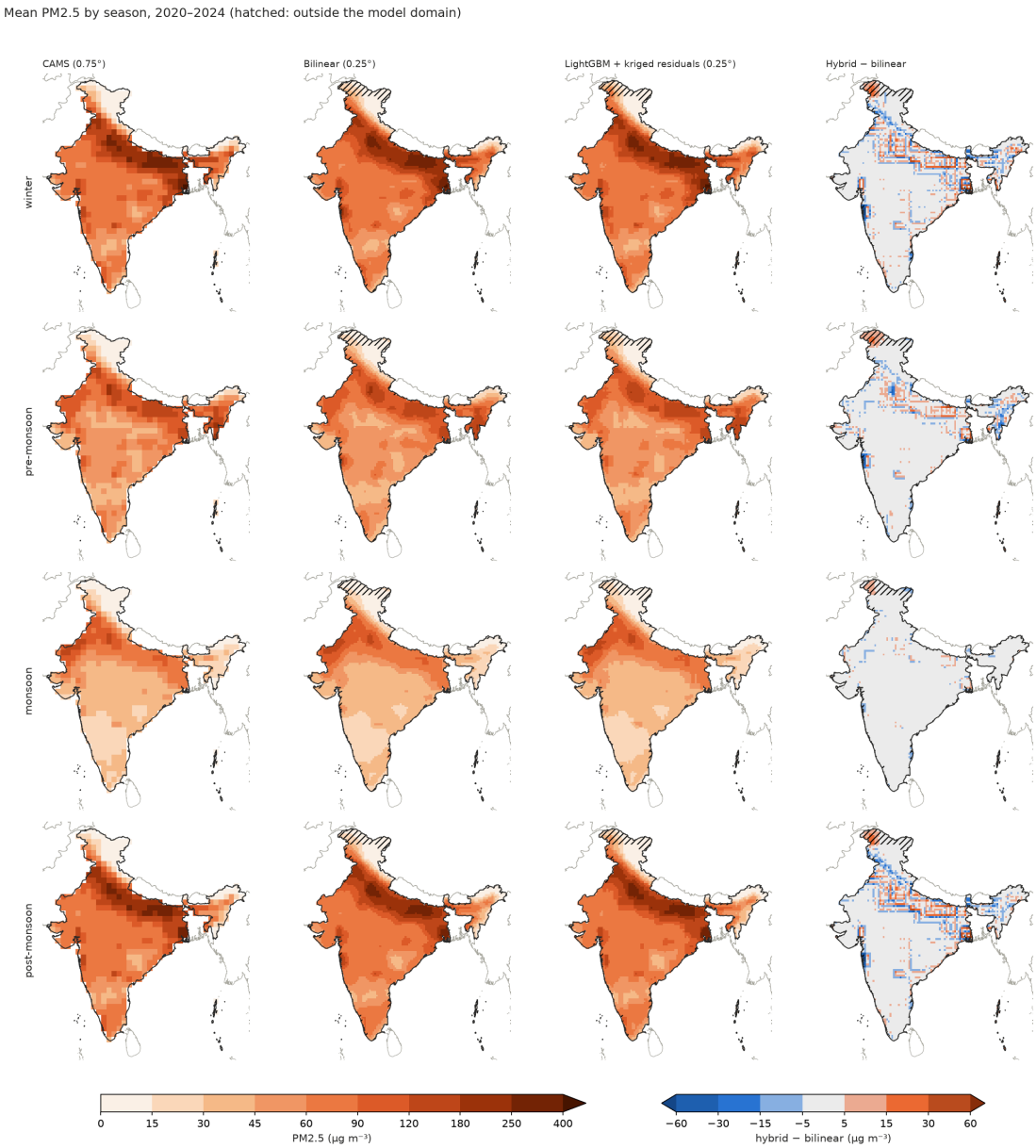


Figure 6: Seasonal mean PM2.5: CAMS, bilinear, hybrid, and their difference (hatched: outside the model domain)

Code: `src/pm25/maps.py`, `scripts/make_maps.py`.

16 Step 12: Writing it up

In one sentence: collect everything into a report, rebuild the final figures, and convert the report to PDF and Word.

The report (`docs/report.md`, also as PDF and Word in `results/12_report/`) answers the six questions with tables and figures. A small script rebuilds the report figures from saved results in seconds, without re-running any model. This booklet is the beginner's companion to that report.

Code: `scripts/report_figures.py`; conversion with *pandoc* and *typst*.

17 The answers in short

Question	Answer
RQ1 How accurately?	At new places about **two thirds of the variation** is explained ($R^2 \approx 0.68$; typical error $\approx 19\ \mu g\ m^{-3}$).
RQ2 Which model?	**LightGBM** (gradient-boosted trees) is best; straight-line models are far behind.
RQ3 Which variables?	**AOD**, then time of day, time of year, location and altitude; wind least.
RQ4 Regions and seasons?	Best in central India and the monsoon; worst in the northeast, the pre-monsoon fire season, at night, and in severe-pollution hours.
RQ5 New places?	Results depend strongly on *which* places are tested; 6 blocks are too few to be sure, 27 give a clearer picture.
RQ6 ML vs interpolation?	ML wins where neighbours aren't representative (mountains, far from data); combining both removes bias and reduces peak errors, but the average gain is small. Finer maps add little that can be verified.

Part III

The code, file by file

18 How the repository is organised

```
pm25-india/
├── CLAUDE.md            rules for working on the project (methods, logging)
├── README.md            one-line description
├── environment.yml      every package needed (conda), incl. this project itself
├── pyproject.toml       makes src/pm25 installable as the package "pm25"
├── src/pm25/            the LIBRARY: reusable functions, no file paths, no plots
├── scripts/             the STEPS: one entry point per pipeline step
├── tests/               automatic checks of the library (pytest)
├── docs/
│   ├── report.md        the project report
│   ├── booklet/         this booklet
│   └── steps/           one document per step, journal, glossary
├── data/                raw downloads and processed data (NOT in git)
├── results/             tables, figures, configs and run logs per step
└── logs/sessions/       archived working sessions (NOT in git)
```

Library vs scripts. The code is split in two on purpose:

- `src/pm25/` holds **small, documented functions** that do one thing each (e.g. "compute inverse-distance weights"). They don't know about file paths or figures, so they are easy to test and reuse.
- `scripts/` holds one **entry point per step** (e.g. `run_baselines.py`). A script reads the inputs, calls library functions, writes results and figures, and records what it did.

How every script is built. All scripts follow the same pattern:

```python
def main() -> None:
    args = parse_args()                          # 1. read command-line options
    setup_logging(log_file=Path("results/logs/run_baselines.log"))
    log_run_info(args)                           # 2. record command, git version, settings
    ...                                          # 3. load data, compute, score
    result.to_csv(args.out_dir / "metrics_overall.csv")   # 4. save tables
    (args.out_dir / "config.json").write_text(json.dumps(config))  # 5. save settings

if __name__ == "__main__":                       # run main() only when started as a script
    main()
```

- `parse_args()` uses Python's **argparse**, so every setting can be changed on the command line (`--seed 42`, `--out-dir` …) and has a sensible default.
- Every run **appends to a log file** in `results/logs/`: the exact command, the git version of the code, every setting and the main results. Together with the `config.json` written next to each step's results, this makes every number traceable to the code and settings that produced it.
- **Random seeds** are fixed (default 42), so random choices (samples, model settings) are the same every time.

The shape of the data. Two layouts appear everywhere:

1. **Gridded**: an *xarray* dataset with dimensions `(time, lat, lon)`, e.g. PM2.5 has shape 14,616 × 43 × 40.
2. **Table of rows**: for ML, each row is one (time step, cell) pair and each column one feature. Rows are **time-major**: all cells of time step 0, then all cells of time step 1, and so on. Keeping this order identical everywhere is what lets predictions be reshaped back into `(time, cell)` arrays.

19 The library: `src/pm25/`

Each section names the file, says what it is for, lists its main functions, and shows a short piece of the real code with an explanation.

19.1 `download.py`: requesting data from Copernicus

Builds one request per month and downloads it safely. Main functions: `iter_months` (all months between two dates), `build_request` (the request for one month), `month_path` (file name such as `eac4_india_2020-01.nc`), `download_month`.

```
def download_month(client, out_dir, year, month):
    target = month_path(out_dir, year, month)
    if target.exists():                        # already done: skip it
        logger.info("Skipping %s (already downloaded)", target.name)
        return target
    partial = target.with_name(target.name + ".part")
    client.retrieve(DATASET, build_request(year, month), str(partial))
    partial.rename(target)                     # only a complete file gets the real name
    return target
```

The `.part` trick means a crash in the middle of a download leaves a `.part` file that the next run ignores and replaces, never a broken file with the final name.

19.2 `regions.py`: India mask and regions

`load_india_states` reads state outlines from Natural Earth; `cell_box` makes the square of one grid cell; `build_region_mask` computes, for every cell, the share of its area inside India, the main state and the region; `check_state_names` stops the program if a state has no region (so a spelling difference can't go unnoticed).

```
for i, la in enumerate(lat):
    for j, lo in enumerate(lon):
        cell = cell_box(float(la), float(lo))
        overlaps = {names[k]: cell.intersection(states[names[k]]).area / cell.area
                    for k in tree.query(cell)}       # only states near this cell
        ...
in_india = fraction >= min_fraction                  # the 50% rule
```

`tree.query` uses an *STRtree*, a spatial index that quickly finds which state shapes are near a cell, instead of testing all 36 states for all 1,720 cells.

19.3 `preprocess.py`: units and derived variables

`standardize` (names and order), `convert_units`, `relative_humidity`, `add_met_features` (wind speed, humidity), `add_time_features` (solar hour, day of year, season), and `preprocess`, which runs them all in order.

```
utc_hour = ds["time"].dt.hour + ds["time"].dt.minute / 60.0
solar_hour = (utc_hour + ds["lon"] / 15.0) % 24.0       # the Sun moves 15° per hour
out["hour_sin"] = np.sin(2 * np.pi * solar_hour / 24.0)
out["hour_cos"] = np.cos(2 * np.pi * solar_hour / 24.0)
```

`% 24.0` wraps the hour around midnight. The sine/cosine pair puts the 24 hours on a circle, so 23:00 and 01:00 end up close together. `hour_cos` is +1 at local midnight and −1 at local noon.

19.4 `exploration.py`: summaries, correlations, variograms

Functions for step 04: `india_cells`, `to_table`, `summarize`, `spearman_with_target`, `haversine_km` (distance between two points on the Earth's surface), `standardize_rows`,

`empirical_variogram`, the variogram models `spherical` and `exponential`, `fit_variogram`, `linear_residuals` (remove what the predictors explain) and `block_size_cells` (block size for a given range).

```
def exponential(h, nugget, psill, rng):
    return nugget + psill * (1.0 - np.exp(-3.0 * h / rng))
```

This is the curve fitted to the variogram: at distance `h = 0` it starts at the *nugget* (variation even between very close cells), and it rises towards `nugget + psill` (the *sill*), reaching 95% of it at the *range* `rng`. `empirical_variogram` computes, for all pairs of cells, half the average squared difference in each distance bin, processing 32 time steps at a time to keep memory use small.

19.5 `splits.py`: blocks, test set and folds

`assign_blocks` gives each cell a block number from its grid position; `region_counts` counts cells per region in each block; `pick_test_blocks` chooses the test blocks; `assign_folds` deals training blocks into 5 balanced folds; `make_split` does everything for one block size; `split_summary` reports the result.

```
for _ in range(n_candidates):                       # try 5,000 random orders
    order = rng.permutation(len(cells))
    cumulative = np.cumsum(per_block[order])
    k = int(np.argmin(np.abs(cumulative / total - test_fraction))) + 1
    chosen = order[:k]                              # first k blocks ≈ 20% of cells
    share = cells[chosen].sum(axis=0) / region_total
    score = np.abs(share - test_fraction).mean()    # how far each region is from 20%
    ...
    if score < best_score:
        best_score, best = score, chosen
```

A simple but effective search: shuffle the blocks, take as many as needed to reach about 20% of all cells, score how evenly the regions are covered, and keep the best of 5,000 tries. Because `rng` is seeded, the same split comes out every time.

19.6 `features.py`: from grids to tables

`stack_cells` turns `(lat, lon)` into one `cell` dimension and keeps only the cells we want; `feature_matrix` builds the `(rows, features)` array; `target_vector` gives PM2.5 in the same row order. `FEATURE_SETS` defines the three feature lists used in step 07 (`brief`, `all`, `all_no_latlon`).

```
FEATURE_SETS = {
    "brief": BRIEF,                       # aod550, u10, v10, t2m, d2m, sp, lat, lon
    "all": BRIEF + DERIVED,               # + wind_speed, rh2m, hour_sin/cos, doy_sin/cos
    "all_no_latlon": [f for f in BRIEF + DERIVED if f not in ("lat", "lon")],
}
```

19.7 `metrics.py`: how every method is scored

`scores` computes RMSE, MAE, R^2, bias and the number of values; `scores_by_group` does it per region, season, etc.; `evaluation_table` builds the long table of (time, test cell) rows with all predictions; `all_scores` runs every grouping.

```
err = pred - obs
sst = np.sum((obs - obs.mean()) ** 2)          # variation around the mean
return {
    "rmse": float(np.sqrt(np.mean(err**2))),
```

```
    "mae": float(np.mean(np.abs(err))),
    "r2": float(1.0 - np.sum(err**2) / sst),   # share of the variation explained
    "bias": float(np.mean(err)),
}
```

Because every step uses this one function, numbers from different steps are directly comparable.

19.8 `baselines.py`: interpolation as weight matrices

`nearest_weights`, `idw_weights`, `delaunay_weights` and `ordinary_kriging_weights` each return a **weight matrix** W with one row per test cell and one column per training cell; `apply_weights` multiplies it with the data.

```
def apply_weights(weights, values_train):
    return values_train @ weights.T        # (time, n_train) @ (n_train, n_test)
```

The key insight: training and test cells are in the same places at every time step, so the weights only need to be computed **once**. Predicting all 14,616 time steps is then a single matrix multiplication (`@`), which takes seconds even for kriging. IDW sorts neighbours with a *stable* sort so that ties (common on a regular grid) are broken the same way on every computer (see chapter 24).

19.9 `models.py`: the ML models and tuning

`build_estimator` creates ridge, random forest, LightGBM or MLP; `ClippedTargetModel` wraps any of them; `make_model` combines the two; `SEARCH_SPACES` lists the settings to try; `sample_params` draws random settings; `fold_indices` and `cross_validate` run block-wise cross-validation; `best_settings` picks the winner per model.

```
class ClippedTargetModel(RegressorMixin, BaseEstimator):
    def fit(self, x, y):
        self.y_max_ = float(np.max(y))
        y_fit = np.log1p(y) if self.target == "log" else y      # optional log target
        self.estimator_ = clone(self.estimator).fit(x, y_fit)
        return self

    def predict(self, x):
        pred = self.estimator_.predict(x)
        if self.target == "log":
            pred = np.expm1(np.clip(pred, None, np.log1p(self.y_max_)))  # undo the log
        return np.clip(pred, 0.0, self.y_max_)                  # never < 0 or > max seen
```

This wrapper gives every model the same two safety features: optionally learning log(1 + PM2.5), and clipping predictions to between 0 and the largest value seen in training (the lesson from the 55,041 µg m^{-3} prediction in step 06).

```
def cross_validate(name, params, x, y, folds, seed):
    per_fold = []
    for _, tr, va in fold_indices(folds):            # each fold = whole blocks
        model = make_model(name, params, seed).fit(x[tr], y[tr])
        per_fold.append(scores(y[va], model.predict(x[va])))
    ...                                              # average over the 5 folds
```

19.10 `hybrid.py`: regression kriging

`out_of_fold_predictions` predicts every training row with a model that did not see its fold; `add_residual_correction` adds interpolated residuals to the ML prediction.

```
def add_residual_correction(ml_test, residual_train, weights):
    return np.clip(ml_test + residual_train @ weights.T, 0.0, None)
```

One line holds the whole hybrid idea: the ML prediction plus the weighted average of nearby errors, clipped so PM2.5 can't become negative.

19.11 `evaluation.py`: bootstrap and CV check

`ist_hour` (Indian time labels), `aqi_level` (India's air-quality categories), `add_time_columns`, `block_sums`, `scores_from_sums`, `block_bootstrap`, `cv_interpolate` (predict each fold from the others) and `nested_oof_residuals`.

```
draws = rng.integers(0, len(blocks), size=(n_boot, len(blocks)))   # 2,000 × 6 picks
counts = np.stack([np.bincount(d, minlength=len(blocks)) for d in draws])
boot = scores_from_sums(np.einsum("rb,bms->rms", counts, cube))
```

A clever shortcut: instead of re-scoring millions of rows 2,000 times, the code first stores, for each block, a few **sums** (number of values, sum of errors, sum of squared errors, …). A bootstrap sample is then just "block 3 twice, block 5 not at all, …", i.e. a weighted sum of those block sums, from which RMSE, MAE, R^2 and bias follow exactly. All 2,000 samples take about a second.

19.12 `importance.py`: SHAP and permutation importance

`FEATURE_GROUPS` lists related variables; `shap_values` asks LightGBM for exact SHAP values; `group_shap` adds them up per group; `mean_abs_by` averages their size; `permutation_importance` shuffles columns and measures the damage; `binned_dependence` makes the "effect vs value" curves.

```
shuffled = x.copy()
perm = rng.permutation(len(x))
shuffled[:, idx] = x[perm][:, idx]       # same shuffle for all columns of a group
s = scores(y, model.predict(shuffled))
```

19.13 `maps.py`: the 0.25° maps

`load_india_official` loads India's official boundary; `fine_axis` builds the fine coordinate axis; `points_inside` keeps grid points inside a shape; `predictors_at_points` interpolates PM2.5 and the predictors to the fine points and derives the extra variables with the **same code as step 03**; `coarse_membership` and `aggregate_to_coarse` average fine values back to the 0.75° cells for the consistency check.

```
points = {"lat": xr.DataArray(lat, dims="cell"), "lon": xr.DataArray(lon, dims="cell")}
out = ds[["pm2p5", *BASE_PREDICTORS]].interp(points, method="linear")
out = add_met_features(out)              # wind speed, humidity (step 03 code)
return add_time_features(out)            # solar hour from each point's longitude
```

19.14 `logging_utils.py`: recording every run

`setup_logging` sends messages to the screen and to a log file; `git_version` finds the current code version (marked `-dirty` if there are unsaved changes); `log_run_info` writes the run header.

```
logger.info("Run started: %s", " ".join(sys.argv))
logger.info("Git version: %s | Python %s", git_version(), sys.version.split()[0])
for name, value in sorted(vars(args).items()):
    logger.info("  %s = %s", name, value)
```

20 The scripts: `scripts/`

Each script is one step of the pipeline. All are run the same way: `conda run -n pm25 python scripts/<name>.py`, from the repository root.

- **`download_eac4.py`** (step 01): downloads every month (skips finished ones), lists failures. Output: `data/raw/eac4/*.nc`.
- **`build_region_mask.py`** (step 02): India mask, states, regions; draws the region map. Output: `data/processed/india_regions.nc`, `results/figures/region_mask.png`.
- **`preprocess_eac4.py`** (step 03): joins 60 months, converts units, adds derived variables. Output: `data/processed/eac4_india_3h.nc`.
- **`explore_eac4.py`** (step 04): summaries, correlations, seasonal maps, daily/monthly cycles, variograms. Output: `results/04_exploration/`.
- **`make_splits.py`** (step 05): 5 × 5 and 8 × 8 block splits, folds, distance to training, maps. Output: `data/processed/splits/`, `results/05_splits/`.
- **`run_baselines.py`** (step 06): nearest, IDW, Delaunay, kriging, linear regression; scores and figures. Output: `results/06_baselines/`.
- **`train_models.py`** (step 07): tunes and trains ridge, RF, LightGBM, MLP; scores on test cells. Output: `results/07_models/<feature set>/`.
- **`run_hybrid.py`** (step 08): out-of-fold residuals, residual variograms, regression kriging. Output: `results/08_hybrid/`.
- **`evaluate.py`** (step 09): all methods together, new breakdowns, block bootstrap, nested CV check. Output: `results/09_evaluation/`.
- **`feature_importance.py`** (step 10): SHAP and permutation importance, heatmaps, dependence curves. Output: `results/10_importance/`.
- **`make_maps.py`** (step 11): 0.25° maps for 4 methods, examples, zoom, consistency check. Output: `results/11_maps/`.
- **`report_figures.py`** (step 12): rebuilds the report figures from saved results (no model runs). Output: `results/12_report/`.
- **`export_session.py`** (tool): archives each working session as raw + readable Markdown. Output: `logs/sessions/`.

A few details worth knowing:

- **`train_models.py`** has two shortcuts: `--cv-from` reuses tuning results already saved (so only the final fits run, ~4 minutes instead of ~40), and `--params-from` reuses chosen settings for another feature set.
- **`run_hybrid.py`** and **`evaluate.py`** *cache* their slowest part (the out-of-fold and nested model fits) in `.nc` files, so a re-run skips it.
- **`make_maps.py`** works **month by month** and only keeps running sums, so the millions of fine-grid predictions never need to be in memory at once.
- **`report_figures.py`** imports plotting functions from `feature_importance.py` and `make_maps.py` instead of copying them, so a figure looks the same whichever script draws it.
- **`export_session.py`** is started automatically when a working session ends (a *hook* in `.claude/settings.json`) and uses only Python's standard library, so it works even without the conda environment.

21 The tests: `tests/`

A **test** is a small program that runs a library function on a tiny example whose correct answer is known, and fails loudly if the answer is wrong. Running `conda run -n pm25 python -m pytest` executes all **73 tests** in a few seconds. Tests were run before every commit, and whenever a bug was found a test was added so that the same mistake can't come back unnoticed.

Test file	Checks, for example, that …
`test_download.py`	month ranges cross year boundaries; leap days are included; finished files are skipped (with a fake client, no internet)
`test_regions.py`	the 50% rule and "largest state wins" work; an unknown state name stops the program
`test_preprocess.py`	units are converted; humidity is lower for drier air; solar hour shifts with longitude; IMD seasons are right
`test_exploration.py`	1° of latitude is ~111 km; a constant field has a zero variogram; a known range is recovered by the fit
`test_splits.py`	a block is never split between train and test or between folds; every region gets test cells
`test_features.py`	table rows are in time-major order and line up with PM2.5
`test_metrics.py`	RMSE, MAE, R^2 and bias give known values; missing values are ignored
`test_baselines.py`	IDW weights sum to 1; Delaunay reproduces a linear field; our kriging matches the PyKrige library; distance ties go to the lower index
`test_models.py`	predictions stay in range; the log target is undone correctly; folds cover every row once; every model fits
`test_hybrid.py`	a fold is never predicted by a model that saw it; the correction is added and clipped
`test_evaluation.py`	bootstrap sums reproduce the plain scores; nested residuals never use the outer fold; IST is UTC + 5:30
`test_importance.py`	SHAP values add up to the model's prediction; the informative feature ranks first
`test_maps.py`	interpolation and derived variables at points; aggregation back to coarse cells
`test_logging_utils.py`	logs record the settings and go to the file
`test_export_session.py`	a session is archived as raw and readable files

A good example of a test that guards a subtle rule:

```
def test_nested_residuals_never_use_the_outer_fold():
    # 3 folds, one cell each, a constant PM2.5 per fold. For outer fold k, the
```

```
    # residual at fold j must come from a model fitted only on the remaining fold.
    ...
    assert np.isnan(resid[0][:, 0]).all()              # the outer fold itself: empty
    assert np.allclose(resid[0][:, 1], 20.0 - 60.0)    # fitted on fold 2 only
```

The data is made so simple that the right answer can be worked out by hand: if any information from the held-out fold leaked into the model, these exact numbers would change and the test would fail.

22 Documentation and record keeping

The project keeps four kinds of records, and all of them are updated **while** working, not afterwards:

1. **Step documents** (`docs/steps/NN_*.md`): one per step, always with the same sections: purpose, idea in plain words, how it works, outputs, how to run, decisions (with reasons), problems met, results, open issues.
2. **The journal** (`docs/steps/journal.md`): a dated timeline of every action, decision, problem and commit, with real times taken from the logs.
3. **Run logs** (`results/logs/*.log`): what each script printed, with the exact command, code version and settings.
4. **Session archives** (`logs/sessions/`): the full working conversations.

git records every change to the code and documents. After each step the work is *committed* (saved as a named version) and *pushed* to GitHub, followed by a small commit that adds the commit itself to the journal. Data files, model files and large prediction files are kept out of git on purpose (they are big and can be regenerated); the `.gitignore` file lists them.

Part IV

Using and judging the project

23 Running the project yourself

23.1 What you need

- A computer with **conda** (Miniforge is a good choice) and about 5 GB of free disk space and 16 GB of memory.
- A free **Copernicus ADS account**. Its API key goes into a file called `~/.cdsapirc` in your home folder. The key is personal: it must never be put into the code or into git.
- **git**, to get the code.

23.2 Setting up

```
git clone git@github.com:dineman/pm25-india.git
cd pm25-india
conda env create -f environment.yml       # creates the "pm25" environment
conda run -n pm25 python -m pytest        # all 73 tests should pass
```

The environment also installs the project's own code (`src/pm25`) as a package, so every script can simply `import pm25`.

23.3 Running the steps

Run from the repository folder, in this order (times measured on a 16 GB Mac):

Step	Command (each starts with `conda run -n pm25 python`)	Time
01	`scripts/download_eac4.py`	~6 h (server queue)
02	`scripts/build_region_mask.py`	seconds
03	`scripts/preprocess_eac4.py`	~1 min
04	`scripts/explore_eac4.py`	~30 s
05	`scripts/make_splits.py`	seconds
06	`scripts/run_baselines.py`	~15 s
07	`scripts/train_models.py --n-iter 8 --cv-time-frac 0.05`	~40 min
08	`scripts/run_hybrid.py`	~9 min
09	`scripts/evaluate.py`	~25 min
10	`scripts/feature_importance.py`	~20 min
11	`scripts/make_maps.py`	~11 min
12	`scripts/report_figures.py`	~10 s

Tips:

- **Try small first.** Step 01 can download a single month (`--start 2020-01 --end 2020-01`), and step 11 can run on a few months (`--max-months 3 --out-dir /tmp/test`).
- **Long runs need a computer that stays awake.** On a laptop, keep it plugged in (chapter 24 tells why).
- **Moving to another computer**: data and model files are not in git. Either copy the `data/` folder (about 1 GB) and re-run steps 06–11, or run everything again.

23.4 Where to find things afterwards

- Numbers: CSV tables in `results/<step>/`.
- Figures: PNG files in the same folders.
- What happened in a run: `results/logs/<script>.log`.
- Why things were done this way: `docs/steps/<step>.md` and the journal.

24 Mistakes we made and what they taught us

Real projects contain mistakes. What matters is finding them, fixing them and recording them honestly. These are the most instructive ones from this project, all documented in the journal.

1. A log-transformed straight line exploded (step 06). Linear regression on log(PM2.5), transformed back with the exponential, predicted 55,041 µg m^{-3} for one hour. *Lesson:* a model can extrapolate wildly outside the data it has seen. *Fix:* every later model clips its predictions to [0, largest value in training].

2. A first test split was unbalanced (step 05). The first version put 49% of the Himalaya into the test set. *Lesson:* check the result of every random procedure, not just that it ran. *Fix:* a search that balances the test share of every region.

3. The wrong "best" settings (step 07, on Fedora). When the tuning tables of the four models were joined, each kept its own row numbers 0–7; looking up "the best row" then returned rows from the wrong model (LightGBM got a setting scoring 40.9 instead of 36.7). The final models trained in that run were invalid. *Lesson:* an innocent-looking table operation can silently mix things up. *Fix:* a dedicated `best_settings` function, a test for it, and a re-run.

4. The same method gave different numbers on two computers (step 09). IDW has no randomness, yet its error was 55.31 on Fedora and 55.16 on the Mac. On a regular grid, the 8th and 9th nearest cells are often exactly equally far away; the sorting routine broke such ties differently on each machine. *Fix:* a *stable* sort, which always keeps the earlier cell, plus a test.

5. Neural networks are not perfectly reproducible. The MLP scored R^2 0.545 on Fedora and 0.528 on the Mac with the same seed, because the underlying number libraries differ. Tree models and interpolation matched exactly. *Lesson:* a fixed seed guarantees the same result on the same machine, not on every machine. The report states which machine produced its numbers.

6. A promising result was partly luck (step 09). The hybrid improved R^2 from 0.59 to 0.72 on the 6 test blocks, but checking on 27 blocks showed the gain came mostly from 2 blocks and is small on average. *Lesson:* with few test areas, always estimate uncertainty and check on more data before believing a big gain.

7. The hybrid map double-counted (step 11). The first version added the *out-of-fold* errors from step 08 to the *final* model. But the final model has already seen the training cells and fits

them well, so adding errors from models that had not seen them over-corrected: pale patches appeared in south India. *Fix:* for a map, the errors must come from the same model that draws the map. A three-month trial run caught this before the full run.

8. The computer fell asleep (step 09). On battery, the Mac went to sleep during the night; 25 minutes of work took 8 hours. *Lesson:* plan long runs on mains power, and always do a short trial run first.

25 What the results can and cannot tell us

- **CAMS is not the real world.** Every "accuracy" here is agreement with a computer model of the atmosphere. CAMS is known to overestimate PM2.5 over the Indo-Gangetic Plain. Real ground measurements would be needed to judge true accuracy.
- **AOD and PM2.5 come from the same model.** In CAMS both are computed from the same simulated particles, so their strong link (and AOD's top importance) is partly built in. With satellite AOD and station PM2.5, the task would be harder.
- **Few test areas.** Six test blocks, mostly one per region, make per-region test results anecdotal. The 27-block check is more reliable, but it slightly favours LightGBM, whose settings were tuned on the same folds.
- **Finer maps can't be checked.** There is no data below 0.75° (80 km) to test the 0.25° maps against; their extra detail comes from what the model learned, not from new measurements. Real enhancement would need fine-resolution inputs (terrain height, satellite AOD at 1–10 km, emission maps) and fine-scale reference data.
- **What is left out.** Cells less than half inside India (including Mumbai and Chennai) and the island territories are not in the training or test data. On the maps, which follow India's official boundary, areas outside the model's domain are hatched.
- **Importance is not cause.** The feature-importance results show what the model does with the data it was given, not what causes pollution.

Within these limits, the project gives a careful, honest answer to its six questions, and every number in it can be traced back to the code and settings that produced it.

26 Glossary

Plain-language meaning of the technical terms used in the step docs. Grouped by topic; the step where a term first matters is in brackets.

26.1 Data

- **CAMS EAC4** (01): the Copernicus Atmosphere Monitoring Service global reanalysis. A computer model of the atmosphere, corrected with satellite observations, giving air quality and weather everywhere on a 0.75° grid (~80 km) every 3 hours. Our "truth" is this model's output, not ground measurements.
- **Reanalysis** (01): a best-estimate reconstruction of past conditions from a model plus all available observations.
- **Grid cell** (02): one 0.75° × 0.75° square of the grid. India has 494 of them.
- **PM2.5** (01): mass of particles smaller than 2.5 μm per m³ of air (μg m^{-3}). The target we predict.
- **AOD** (01): Aerosol Optical Depth, how much the aerosol in the whole air column dims sunlight. No units. Related to PM2.5, but PM2.5 is at the surface.

- **u10, v10** (01): east-west and north-south wind at 10 m (m s^{-1}).
- **t2m, d2m** (01): temperature and dewpoint at 2 m. Dewpoint = temperature at which the air would become saturated; close to t2m means humid air.
- **Relative humidity (RH)** (03): how close the air is to saturation, in %. Computed from t2m and d2m.
- **Surface pressure (sp)** (03): mostly tells altitude: ~1,000 hPa at sea level, ~530 hPa in Ladakh.
- **IMD seasons** (03): India Meteorological Department seasons: winter (Jan–Feb), pre-monsoon (Mar–May), monsoon (Jun–Sep), post-monsoon (Oct–Dec).
- **Local solar hour** (03): clock time set by the sun at that longitude (noon = sun highest). Used instead of IST because the sun rises ~2 h earlier in the east of India than in the west.
- **sin/cos encoding** (03): writing a cyclic value (hour, day of year) as two numbers sin and cos, so 23:00 and 01:00 come out close together.
- **IGP** (02): Indo-Gangetic Plain, the flat, densely populated north (Punjab to West Bengal); the most polluted region.

26.2 Statistics and spatial analysis

- **Percentile** (03): the value below which that % of the data falls. Median = 50th percentile.
- **Skewed distribution** (03): most values small, a few very large (PM2.5: median 52, max 5,651). Means are pulled up by the large ones.
- **log1p transform** (03): log(1 + x). Squeezes large values together so the extremes dominate less; the +1 handles zeros. Undo with exp(x) − 1 (`expm1`).
- **Spearman correlation** (04): correlation of the *ranks* of two variables, from −1 to 1. Measures whether one tends to go up when the other does, even if not in a straight line; insensitive to extreme values.
- **Variogram** (04): how different two cells' values are, on average, as a function of the distance between them. Semivariance $\gamma(h) = \frac{1}{2} \cdot$ mean of (difference)^2 for pairs at distance h. Small γ = similar values.
- **Nugget** (04): variogram value at (almost) zero distance: variation that even neighbouring cells don't share (noise, very local effects).
- **Sill** (04): the level where the variogram flattens: cells this far apart are no more alike than random pairs.
- **Range** (04): the distance at which the sill is (practically) reached, i.e. how far spatial correlation reaches. For the exponential model we use the *practical* range (95% of sill).
- **Detrending** (04): removing a smooth large-scale pattern (here: the north-south gradient, via a lat/lon polynomial) before measuring local correlation. Without it, the variogram never flattens.
- **Residual** (04): observed value minus what a model predicts; the part the model can't explain.
- **Standardise** (04): subtract the mean and divide by the standard deviation, so the values have mean 0 and spread 1.

26.3 Splitting and evaluation

- **Train / test split** (05): training cells are used to build a model; test cells are hidden until the very end and used once to measure how well it works on locations it has never seen.

- **Spatial block split** (05): whole squares of cells (blocks) go to test together, so test cells are not right next to training cells. Random single-cell hold-out would give over-optimistic scores, because neighbours are nearly copies of each other.
- **Cross-validation (CV)** (05, 07): repeat "fit on some training data, score on the rest" several times with different parts held out, and average the scores. Used to *choose* settings without touching the test set.
- **Fold** (05): one of the parts training cells are divided into for CV. With 5 folds, each fold is held out once while the other 4 are used for fitting.
- **Block-wise (grouped) CV** (07): CV where whole blocks are kept together in a fold, for the same reason as the spatial test split.
- **Data leakage** (05, 07): information from the test set influencing training or choices, which makes scores look better than they really are.
- **RMSE** (06): root mean squared error = √mean(error²), in μg m⁻³. Punishes big errors heavily, so a few extreme values (fire peaks) dominate it.
- **MAE** (06): mean absolute error = mean(|error|), in μg m⁻³. The typical size of an error; less sensitive to extremes than RMSE.
- **R^2** (06): 1 − (squared error of the model) / (squared error of always predicting the mean). 1 = perfect, 0 = no better than the mean, negative = worse than the mean.
- **Bias** (06): mean(prediction − observation). Negative = predicts too low on average.
- **Bootstrap** (09): estimate how much a result could vary by re-drawing the data *with replacement* many times and recomputing it each time.
- **Block bootstrap** (09): bootstrap that re-draws whole spatial blocks, not single rows, because neighbouring cells are not independent.
- **95% range (percentile interval)** (09): the middle 95% of the bootstrap results; a rough "plausible range" for the score.
- **Paired comparison** (09): comparing two methods on exactly the same re-drawn blocks, so the difference isn't blurred by which blocks were drawn.
- **Nested CV** (09): CV inside CV. Used when a method itself needs out-of-fold values (the hybrid's residuals), so that the fold being scored is never seen by any model involved in predicting it.
- **AQI categories** (09): India's National Air Quality Index bands for PM2.5 (good < 30, satisfactory 30–60, moderate 60–90, poor 90–120, very poor 120–250, severe ≥ 250 μg m⁻³, defined on 24 h means).
- **IST** (09): Indian Standard Time = UTC + 5:30. The CAMS time steps 00, 03, …, 21 UTC are 05:30, 08:30, …, 02:30 IST.
- **Boundary layer** (09): the lowest part of the atmosphere, mixed by the ground's heating. Deep in the afternoon (pollution spread out), shallow at night (pollution trapped near the ground).

26.4 Interpolation (baselines)

- **Spatial interpolation** (06): estimating a value at an unobserved place from values at nearby observed places, using only distance and position.
- **Nearest neighbour** (06): copy the value of the closest observed cell.
- **IDW** (06): inverse-distance weighting, a weighted average of nearby cells, closer cells weighted more (weight 1/distance²).

- **Bilinear / Delaunay-linear** (06): bilinear blends the 4 surrounding grid points; it needs a complete grid. With holes in the grid we use the scattered version: join observed cells into triangles (Delaunay triangulation) and blend the 3 corners of the triangle containing the point.
- **Ordinary kriging** (06): a weighted average whose weights come from the variogram, chosen to be the best unbiased estimate given how correlation decays with distance. Also handles clustering of observed points ("screening": nearer points hide those behind them).
- **Regression kriging** (08): predict with a regression/ML model, then krige the model's residuals from nearby training cells and add them back.

26.5 Machine learning (step 07)

- **Feature / predictor** (07): an input variable (AOD, wind, ...). **Target**: what we predict (PM2.5).
- **Hyperparameter / setting** (07): a choice made *before* training that controls how a model learns (e.g. number of trees, network size), as opposed to what it learns from the data.
- **Tuning** (07): trying several hyperparameter settings and keeping the one with the best CV score.
- **Random search** (07): tuning by trying a fixed number of randomly chosen settings (with a fixed seed so it is repeatable), instead of every combination.
- **Seed** (all): starting number for random choices, fixed so results can be reproduced exactly.
- **Overfitting** (07): a model learns details of its training data that don't carry over to new data; it scores well in training and worse on test.
- **Extrapolation** (06, 07): predicting outside the range of the training data. Linear models and neural networks can produce absurd values there; tree models can't go beyond the values they have seen.
- **Clipping** (07): forcing predictions into an allowed range, here 0 to the highest PM2.5 seen in training.
- **Ridge regression** (07): linear regression with a penalty (alpha) that keeps coefficients small, making it more stable when features are correlated.
- **Decision tree** (07): a flowchart of yes/no questions on features ("is AOD > 0.8?") ending in a predicted value.
- **Random forest** (07): many decision trees, each built on a random sample of rows and features; prediction = their average. Robust, few settings to tune.
- **Gradient boosting / LightGBM** (07): trees built one after another, each correcting the errors of the ones before. LightGBM is a fast implementation. Usually the strongest model on tabular data.
- **Learning rate** (07, boosting/MLP): how big each correction step is. Smaller = slower but often more accurate.
- **MLP (multilayer perceptron)** (07): a small neural network: layers of units, each computing a weighted sum of its inputs passed through a simple non-linear function. Needs features scaled to similar ranges.
- **Early stopping** (07): stop training a neural network when the score on a held-out part stops improving, to avoid overfitting.

26.6 Feature importance (step 10)

- **Feature importance** (10): how much a model relies on each feature.

- **SHAP value** (10): for one prediction, the share contributed by one feature, measured against the model's average prediction. A prediction's SHAP values add up to the prediction minus the average. Comes from game theory (Shapley values): a fair way of sharing a total among players.
- **TreeSHAP** (10): an exact, fast way to compute SHAP values for tree models; LightGBM has it built in (`pred_contrib=True`).
- **Mean |SHAP|** (10): average size of a feature's contribution, ignoring sign; the usual SHAP importance score. In log units here: 0.1 ≈ ±10% PM2.5.
- **Permutation importance** (10): shuffle one feature's values between rows and measure how much the error grows. Big growth = the accuracy depends on it.
- **Grouped permutation** (10): shuffle related features together (e.g. temperature, dewpoint, humidity), so they can't stand in for each other.
- **Dependence curve** (10): average SHAP value of a feature plotted against its value: shows *how* the feature acts (rising, falling, threshold).

26.7 Maps (step 11)

- **Regridding** (11): moving a gridded field to another grid, e.g. from 0.75° to 0.25°. Bilinear regridding blends the 4 surrounding values.
- **Downscaling / spatial enhancement** (11): producing a finer map than the input data. Statistical downscaling uses a model to add detail; without fine-scale inputs or reference data the added detail can't be checked.
- **Consistency check** (11): averaging a fine map back to the coarse grid and comparing with the original coarse data. It tests faithfulness, not accuracy at the fine scale.
- **Point-of-view boundaries** (11): Natural Earth versions of country borders as officially claimed by one country. The India version includes all of Jammu & Kashmir and Ladakh (PoK, Gilgit-Baltistan, Aksai Chin).